UNIT 1: INTRODUCTION TO PYTHON AND JUPYTER NOTEBOOK

Overview Of Languages

Procedure Oriented Language:

- Procedural programming uses a list of instructions to tell the computer what to do step-by-step.
- Procedures, also known as routines, or functions.
- If you want a computer to do something, you should provide step-by-step instructions to computer, how to do it.
- Examples of procedural languages include FORTRAN, COBOL, C and PASCAL.

Object Oriented Language:

- Object Oriented programming (OOP) is a programming language that relies on the concept of classes and objects.
- A Class in object-oriented programming is a blueprint or prototype that defines the variables and the methods (functions) common to Objects of a certain kind.
- An object in OOPS is a specimen of a class.
- **Example:**
- Class: Car (Attributes: Color, Brand, Model and Methods: repaint())
- Object: My Car (Attributes: Color = Blue, Brand = Alto, Model = Charger

  Methods: repaint())
- Object: My Friend's Car (Attributes: Color = Red, Brand = Nissan, Model = Ultima

  Methods:repaint())
- There are many object-oriented programming languages including JavaScript, C++, Java.

Machine Level Language:

- The machine-level language is a language that consists of a set of instructions that are in the binary form 0 or 1 which is also known as binary language or low level language.
- As we know that computers can understand only machine instructions, which are in binary digits, i.e., 0 and 1, so the instructions given to the computer can be only in binary codes.
- Creating a program in a machine-level language is a very difficult task as it is not easy for the programmers to write the program in machine instructions.
- It is error-prone as it is not easy to understand, and its maintenance is also very high.

Interpreter:

- Python interpreter is software logic between code and computer hardware. It takes a python program (like *.py file) and run the program line by line from start to bottom.

Compiler:

- A compiler is a computer program which helps you transform source code written in a high-level language into low-level machine language.It takes entire program at a time.

Introduction to Python:

- Python is a general-purpose high-level programming language.
- Python was developed by Guido Van Rossam in 1989 while working at National Research Institute at Netherlands.
- But officially Python was made available to public in 1991. The official Date of Birth for Python is : Feb 20th 1991.
- Python is recommended as first programming language for beginners.
- In various books of python programming, it is mentioned that python language is interpreted. But that is half correct the python program is first compiled and then interpreted. The compilation part is hidden from the programmer thus, many programmers believe that it is an interpreted language. The compilation part is done first when we execute our code and this will generate byte code and internally this byte code gets converted by the python virtual machine(p.v.m) according to the underlying platform(machine+operating system).
- python -m dis xyz.py (In cmd we can see compiled bytecode of xyz.py file using this command).

Example1: To print Hello world

C:

```
#include<stdio.h>
void main()

{
printf("Hello world");
}
```

Java:

```
class Test

{

    public static void main(String []args)

    {

        System.out.println("My First Java Program.");

    }

};
```

Python:

```
print("Hello World")
```

- The name Python was selected from the TV Show "The Complete Monty Python's Circus", which was broadcasted in BBC from 1969 to 1974.
- Guido developed Python language by taking almost all programming features from different languages

(1) Functional Programming Features from C
(2) Object Oriented Programming Features from C++

(3) Scripting Language Features from Perl and Shell Script

(4) Modular Programming Features from Modula-3.Most of syntax in Python Derived from C and ABC languages.

Where we can use Python?

We can use everywhere. The most common important application areas are

- Desktop Applications
- Web Applications
- Database Applications
- Network Programming
- Developing games
- Data Analysis Applications
- Machine Learning
- Artificial Intelligence Applications
- IoT

Features of Python:

Simple and easy to learn: Python is a simple programming language.
When we read Python program, we can feel like reading English statements.
The syntaxes are very simple and only 30+ keywords are available.
When compared with other languages, we can write programs with very a smaller number of lines. Hence more readability and simplicity.

Freeware and Open Source: We can use Python software without any license and it is open source.

High Level Programming language: Python is high level programming language and henceit is programmer friendly language.

Being a programmer, we are not required to concentrate low level activities like memory management and security etc.

Platform Independent: Once we write a Python program, it can run on any platform without rewriting once again.

Internally PVM is responsible to convert into machine understandable form.

Portability: Python programs are portable. ie we can migrate from one platform to another platform very easily.

Python programs will provide same results on any platform.
Dynamically Typed: In Python we are not required to declare type for variables.

Whenever we are assigning the value, based on value, type will be allocated automatically.
Both Procedure Oriented and Object Oriented: Python language supports both Procedure oriented (like C, pascal etc) and object oriented (like C++, Java) features.

Hence, we can get benefits of both like security and reusability etc.
Interpreted: We are not required to compile Python programs explicitly.
Internally Python interpreter will take care that compilation. If compilation fails interpreter raised syntax errors.

Once compilation success then PVM (Python Virtual Machine) is responsible to execute.
Extensible: We can use other language programs in Python.

Embedded: We can use Python programs in any other language programs. i.e. we can embed Python programs anywhere.

Extensive Library: Python has a rich inbuilt library.
Being a programmer, we can use this library directly and we are not responsible to implementthe functionality.

Jupyter Notebook:

- A Jupyter notebook is a file that contains both programming code and text descriptions.
- Jupyter notebooks can also contain embedded charts, plots, images, videos, and links.
- Jupyter notebooks run in a web browser like Firefox or Google Chrome.
- A Jupyter notebook is an application that can run Python code, display plots, show equations and contain formatted text.

Installing Jupyter Notebook:

- Jupyter installation requires Python 3.3 or greater version.
- As an existing Python user, you may wish to install Jupyter using Python's package manager, pip.
- First, ensure that you have the latest pip; older versions may have trouble with some dependencies:
- pip3 install --upgrade pip
- Then install the Jupyter Notebook using: pip install jupyter.

Opening and Creating Jupyter Notebook:

- Follow the steps given below to open the Jupyter notebook:
- Open the juputer on Windows start menu and select --> [Jupyter]
- This action opens the Jupyter file browser in a web browser tab.
- In the upper right select [New] --> [Python 3]
- A new notebook will open as a new tab in your web browser.
- Try typing the code below in the first cell in the notebook to the right of the In [ ]: prompt:
- Then click the run button in the middle of the menu at the top of the notebook.

Unit 1: Introduction To Python And Jupyter Notebooks

Saving and downloading Jupyter Notebook:

- Jupyter notebooks can be saved using the save icon in the upper menu or by pressing [Ctrl] + [s].
- Jupyter notebooks can also be saved as a copy, similar to the Save As command common in many programs. To make a copy of a Jupyter notebook use File --> Make a Copy.
- Jupyter notebooks can be downloaded and saved using File --> Download As --> Notebook (.ipynb). Selecting this menu option will download the notebook as a .ipynb file.
- Jupyter notebooks can be saved in other formats besides the native .ipynb format.
- These formats can be accessed using the [File] --> [Download As] menu.
- The available file download types are:
- Notebook (.ipynb) - The native jupyter notebook format
- Python (.py) - The native Python code file type.
- HTML (.html) - A web page
- Markdown (.md) - Markdown format
- reST (.rst) - Restructured text format
- LaTeX (.tex) - LaTeX Article format
- PDF via LaTeX (.pdf) - a pdf exported from LaTeX, requires a converter

Python - Shell:

- Python is an interpreter language. It means it executes the code line by line. Python provides a Python Shell, which is used to execute a single Python command and display the result.
- It is also known as REPL (Read, Evaluate, Print, Loop), where it reads the command, evaluates the command, prints the result, and loop it back to read the command again.
- To run the Python Shell, open the command prompt or power shell on Windows write python and press enter. A Python Prompt comprising of three greater-than symbols >>> appears.
- Now, you can enter a single statement and get the result. For example, enter a simple expression like 3 + 2 press enter and it will display the result in the next line.

Execute Python Script:

- As you have seen above, Python Shell executes a single statement. To execute multiple statements, create a Python file with extension .py, and write Python scripts (multiple statements).
- For example, enter the following statement in a text editor such as Notepad.
- print ("This is Python Script.")
- print ("Welcome to Python Tutorial")
- Save it as myPythonScript.py, navigate the command prompt to the folder where you have saved this file and execute the python myPythonScript.py command. It will display the result.

Python - IDLE:

- IDLE (Integrated Development and Learning Environment) is an integrated development environment (IDE) for Python.
- The Python installer for Windows contains the IDLE module by default.
- To start an IDLE interactive shell, search for the IDLE icon in the start menu and double click on it.
- This will open IDLE, where you can write and execute the Python scripts.
- You can execute Python statements same as in Python Shell.
- To execute a Python script, create a new file by selecting File -> New File from the menu.
- Enter multiple statements and save the file with extension .py using File -> Save. For example, save the following code as MyProgram.py.
- print (“Hello World")
- print ("Welcome to Python Tutorial")
- Press F5 to run the script in the editor window. The IDLE shell will show the output.

Python - IDEs:

- There are many free and commercial IDEs available for Python.
- Jupyter Notebook, Visual Studio Code, Spyder, PyCharm

IDENTIFIERS:

- A Name in Python Program is called Identifier.
- It can be Class Name OR Function Name OR Module Name OR Variable Name.

  a = 10

Rules to define Identifiers in Python:

(1) The only allowed characters in Python are:

- alphabet symbols (either lower case or upper case)
- digits (0 to 9)
- underscore symbol (_)
- By mistake if we are using any other symbol like $ then we will get syntax error.
- cash = 10 → Valid
- ca$h =20 → Invalid

(2) Identifier should not starts with digit.

- 123total → Invalid
- total123 → Valid

(3) Identifiers are case sensitive. Of course, Python language is case sensitive language.

- total=10
- TOTAL=999
- print(total) #10
- print(TOTAL) #999

(4) We cannot use reserved words as identifiers

- def = 10 → Invalid

(5) There is no length limit for Python identifiers. But not recommended to use too lengthy identifiers.

(6) If Identifier is start with Underscore (_) then it indicates it is private.

(7) If identifier starts with __(Two Under Score Symbols) indicating that strongly private identifier.

(8) If the identifier starts and ends with two underscore symbols then the identifier is language defined special name, which is also known as magic methods.

Eg: __add____

❖ **Reserved Words or Keywords:**

- In Python some words are reserved to represent some meaning or functionality. Such types of words are called reserved words.
- There are 33 reserved words available in Python.
- True, False, None, and, or ,not,is,if, elif, else, while, for, break, continue, return, in, yield, try, except, finally, raise, assert, import, from, as, class, def, pass, global, nonlocal, lambda, del, with.
- All Reserved words in Python contain only alphabet symbols.
- Except the following 3 reserved words, all contain only lower case alphabet symbols.
- True
- False
- None
- **Example:**
- a= true ×
- a=True √

Data Types:

Python has the following data types built-in by default, in these categories:

- **Text Type:** str
- **Numeric Types:** int, float, complex
- **Sequence Types:** list, tuple, range

- **Mapping Type:** dict
- **Set Types:** set, frozenset
- **Boolean Type:** bool
- **Binary Types:** bytes, bytearray

Text Type (str):

- str represents String data type.
- A String is a sequence of characters enclosed within single quotes or double quotes.
- s1=‘Arman’ or s1=“Arman"
- By using single quotes or double quotes we cannot represent multi line string literals.
- For this requirement we should go for triple single quotes (''') or triple double quotes (""")
- s1=′′′Arman
- soft'''
- s1="""Arman
- soft"""
- We can display a string literal with the print() function:
- print("Hello") # Hello
- print('Hello’) # Hello
- Assigning a string to a variable is done with the variable name followed by an equal sign and the string:
- a = ′′Arman′′
- print(a) # Arman

Numeric Type:

- There are three numeric types in Python:

(1) int: Int, or integer, is a whole number, positive or negative, without decimals, of unlimited length.

Example:

```
x = 1
y = 35656333554887711
z = -3255544
print(type(x)) # <class ‘int’>
print(type(y)) # <class ‘int’>
print(type(z)) # <class ‘int’>
```

We can represent int values in the following ways:

1) Decimal form 2) Binary form 3) Octal form 4) Hexa decimal form

I) Decimal Form (Base-10):

It is the default number system in Python

The allowed digits are: 0 to 9. Eg: a =10

II) Binary Form (Base-2):

The allowed digits are : 0 & 1

Binary value should be prefixed with 0b or 0B

Eg: a = 0B1111(Valid) , a = 0B123(Invalid)

III) Octal Form (Base-8):

The allowed digits are : 0 to 7

Octal value should be prefixed with 0o or 0O.

Eg: a = 0o123(Valid) , a = 0o786(Invalid)

IV) Hexa Decimal Form (Base-16):

The allowed digits are: 0 to 9, a-f (both lower and upper cases are allowed)

Hexadecimal value should be prefixed with 0x or 0X

Eg: a = 0XFACE, a = 0XBeef, a = 0XBeer

Being a programmer, we can specify literal values in decimal, binary, octal and hexa decimal forms. But PVM will always provide values only in decimal form.

Example:

```
a=10
b=0o10
c=0X10
d=0B10
print(a) #10
print(b) #8
print(c) #16
print(d) #2
```

Base Conversions:

1) bin(): We can use bin() to convert from any base to binary.

```
>>> bin(15)  # '0b1111'
>>> bin(0o11)  # '0b1001'
>>> bin(0X10)  # '0b10000'
```

2) oct(): We can use oct() to convert from any base to octal.

```
>>> oct(10)  # '0o12'
>>> oct(0B1111)  # '0o17'
>>> oct(0X123)  # '0o443'
```

3) hex(): We can use hex() to convert from any base to hexa decimal.

```
>>> hex(100)  # '0x64'
>>> hex(0B111111)  # '0x3f'
>>> hex(0o12345)  # '0x14e5'
```

float: Float, or "floating point number" is a number, positive or negative, containing oneor more decimals.

Example:

```
x = 1.20
y = 1.00
z = -34.58
print(type(x)) <class ‘float’>
print(type(y)) <class ‘float’>
print(type(z)) <class ‘float’>
```

Float can also be scientific numbers with an "e" to indicate the power of 10.

Example:

```
x = 36e3
y = 11E4
z = -87.6e100
print(type(x)) <class ‘float’>
print(type(y)) <class ‘float’>
print(type(z)) <class ‘float’>
```

Boolean Type (bool):

Booleans represent one of two values: True or False.

In programming you often need to know if an expression is True or False.

You can evaluate any expression in Python, and get one of two answers, True or False.

When you compare two values, the expression is evaluated and Python returns the Boolean answer.

When you run a condition in an if statement, Python returns True or False.

Almost any value is evaluated to True if it has some sort of content.

Any string is True, except empty strings and 0

Example:

```
print(20 > 8) # True
print(20 == 8) # False
print(20 < 8) # False
print(bool("abc")) # True
print(bool("")) # False
print(bool(123)) # True
print(bool(["apple", "cherry", "banana"])) # True
print(bool(0)) # False
```

- You can get the data type of a variable with the **type() function.**

Example:

```
x = 3
print(type(x))  #<class 'int'>
y = "Sai"
print(type(y))  #<class 'str'>
```

- Python allows you to assign values to multiple variables in one line. It is called assignment of many values to multiple variables.

Example:

```
a, b, c = "Orange", "Banana", "Cherry"
print(a) # Orange
print(b) # Banana
print(c) # Cherry
```

- Python allows you to assign the same value to multiple variables in one line. It is called assignment of one value to multiple variable.

Example:

```
a=b=c= "Apple"
print(a)  # Apple
print(b)  # Apple
print(c) # Apple
```

- Python allows you to combine both text and a variable, Python uses the + character for this purpose.

Example:

```
a ="awesome"
```

print("Python is "+ a) # Python is awesome.

For numbers, the + character works as a mathematical operator.

Example:

a=20

b=30

print(a+b) # 50

Python will give you an error, if you will combine string and a number.

Example:

x=30

y="$Arman$"

print(x+y) # TypeError: unsupported operand type(s) for +: 'int' and 'str'

Comments:

Comments can be used to explain Python code.

Comments can be used to make the code more readable.

Comments starts with a # , and Python will ignore them.

Example:

#This is a comment

print("Hello, Aayush") # Hello, Aayush

Comments can be placed at the end of a line, and Python will ignore the rest of the line.

Example:

print("Hello, Aayush") #This is a comment

A comment does not have to be text that explains the code, it can also be used to prevent Python from executing code

Example:

#print("Hello, Aayush")

print("Hello, World") # Hello, World

Python does not really have a syntax for multi line comments.

To add a multiline comment you could insert a # for each line

Example:

#This is a comment

#written in

#more than just one line

print("Hello, Aayush!") # Hello, Aayush

Since Python will ignore string literals that are not assigned to a variable, you can adda multiline string (triple quotes) in your code, and place your comment inside it.

Example:

```
"""
This is a comment
written in
more than just one line
"""
print("Hello, Aayush") # Hello, Aayush
```

Reading Input from Users:

Python allows for user input.

That means we are able to ask the user for input.

Python 3.6 uses the input() method.

Example:

```
username = input("Enter username:") # Enter username: Arman
print("Username is: " + username) # Username is: Arman
```

TYPE CASTING:

We can convert one type value to another type. This conversion is called Typecastingor Type conversion.

The following are various inbuilt functions for type casting.

1) int() 2) float() 3) bool() 4) str()

int(): We can use this function to convert values from other types to int.

```
>>> int(123.987)  # 123
>>> int(True)  #1
>>> int(False)   # 0
>>> int("10")   #10
>>> int("10.5")  #ValueError: invalid literal for int() with base 10: '10.5'
>>> int("ten")  #ValueError: invalid literal for int() with base 10: 'ten'
>>> int("0B1111")  #ValueError: invalid literal for int() with base 10: '0B1111'
```

float(): We can use float() function to convert other type values to float type.

```
>>> float(10)   #10.0
>>> float(True) #1.0
>>> float(False) #0.0
```

```
>>> float("10") #10.0
>>>float("10.5") #10.5
>>> float("ten") # ValueError: could not convert string to float: 'ten'
>>> float("0B1111")  # ValueError: could not convert string to float: '0B1111'
```

bool(): We can use this function to convert other type values to bool type.

```
bool(0)  # False
bool(1)  #True
bool(10)  #True
bool(10.5)  #True
bool(0.178)  #True
bool(0.0)  #False
bool(10-2j)  #True
bool(0+1.5j)  #True
bool(0+0j)  #False
bool("True")   #True
bool("False")  #True
bool("")  #False
```

str(): We can use this method to convert other type values to str type.

```
>>> str(10)  # '10'
>>> str(10.5)  #'10.5'
>>> str(10+5j)  # '(10+5j)'
>>> str(True)  #'True'
```

Python Operators:

Operator is a symbol that performs certain operations.Python provides the following set of operators:

- Arithmetic Operators
- Relational Operators OR Comparison Operators
- Logical operators
- Ternary Operators OR Conditional Operators
- Bitwise operators (not in syllabus LJU)
- Assignment operators
- Identity Operators (not in syllabus LJU)
- Membership Operators

Arithmetic Operators:

+	→	Addition (It is also used for string concatenation)
-	→	Subtraction
*	→	Multiplication (It is also used for string multiplication)
/	→	Division
%	→	Modulus
//	→	Floor Division
**	→	Exponent OR Power

Example:

```
a=10
b=2
print('a+b=',a+b)          # a+b = 12
print('a-b=',a-b)          # a-b= 8
print('a*b=',a*b)          # a*b= 20
print('a/b=',a/b)          # a/b= 5.0
print('a//b=',a//b)        # a//b= 5
print('a%b=',a%b)          # a%b= 0
print('a**b=',a**b)        # a**b= 100
```

/ operator always performs floating point arithmetic. Hence it will always returns floatvalue.

But Floor division (//) can perform both floating point and integral arithmetic. If arguments are int type then result is int type. If at least one argument is float type thenresult is float type.

We can use +,* operators for str type also. If we want to use + operator for str type then compulsory both arguments should bestr type only otherwise we will get error.

Example:

```
“Arman"+10  # TypeError: must be str, not int
“Arman"+"10" # ‘Arman10’
```

If we use * operator for str type then compulsory one argument should be int andother argument should be str type.

Example:

```
2*“Arman"  OR # ArmanArman
“Arman"*2  # ArmanArman
2.5*“Arman" # TypeError: can't multiply sequence by non-int of type 'float’
“Arman"*“Arman" # TypeError: can't multiply sequence by non-int of type 'str'
```

Relational Operators:

>	→	Greater than
<	→	Less than
>=	→	Greater than or equal to
<=	→	Less than or equal to
==	→	Equal to
!=	→	Not equal to

Example:

```
a=10
b=20
print("a > b is ",a>b)  # a > b is  False
print("a >= b is ",a>=b) # a >= b is  False
print("a < b is ",a<b)  # a < b is  True
print("a <= b is ",a<=b)  # a <= b is  True
```

We can apply relational operators for str and bool types also.

Example:

```
a="Arman"
b="Arman"
print("a > b is ",a>b)  # a > b is  False
print("a <= b is ",a<=b)  # a <= b is  True
print(True>=True)  # True
print(10 >True)  # True
print(False > True)  # False
```

Chaining of relational operators is possible. In the chaining, if all comparisons returnsTrue then only result is True. If atleast one comparison returns False then the result isFalse

```
10<20 # True
10<20<30 # True
10<20<30<40  # True
10<20<30<40>50  # False
```

We can apply == and != operators for any type even for incompatible types also.

```
10==20  # False
10!= 20  # True
10==True  # False
False==False  # True
```

“Arman"==“Arman" # True

Chaining concept is applicable for equality operators. If atleast one comparisonreturns False then the result is False. Otherwise the result is True.

10==20==30==40 # False

10==10==10==10 # True

Logical Operators :

We can apply logical operators (and,or,not) for all types.

For Boolean Types Behaviour:

- and : If both arguments are True then only result is True.
- or: If atleast one arugemnt is True then result is True.
- not: Complement

Example:

True and False #False

True or False #True

not False #True

For Non-Boolean Types Behaviour:

0 means False

non-zero means True

Empty string is always treated as False

x and y: If x is evaluates to false return x otherwise return y

Example:

10 and 20 # 20

0 and 20 # 0

If first argument is zero then result is zero otherwise result is y

x or y: If x evaluates to True then result is x otherwise result is y

Example:

10 or 20 # 10

0 or 20 # 20

not x: If x is evaluates to False then result is True otherwise False

Example:

not 10 # False
not 0 # True

Ternary OR Conditional Operators :

Syntax: x = first Value if condition else second Value

If condition is True then first Value will be considered else second Value will be considered.

Example:

```
a,b=10,20
x=30 if a<b else 40
print(x)  # 30
```

Assignment Operators:

We can use assignment operator to assign value to the variable.

Example:

```
x = 10
```

We can combine assignment operator with some other operator to form compound assignment operator.

Example:

x += 10 → x = x+10

The following is the list of all possible compound assignment operators in Python.

- += , -= , *= , /= , %= , //= , **= , &= , |= , ^= , >>= , <<=

Example:

```
x=10
x+=20
print(x)  # 30
```

Identity Operators:

We can use identity operators for address comparison.

There are 2 identity operators.

is

is not

r1 is r2 returns True if both r1 and r2 are pointing to the same object.

r1 is not r2 returns True if both r1 and r2 are not pointing to the same object.

Example:

```
a=10
b=10
print(a is b)  # True
```

```
x=True
y=True
print( x is not y)  # False
```

Note: We can use is operator for address comparison where as == operator for content comparison.

Membership Operators:

We can use Membership operators to check whether the given object present in thegiven collection. (It may be String, List, Set, Tuple OR Dict)

in → Returns True if the given object present in the specified Collection.

not in → Returns True if the given object is not present in the specified Collection.

Example:

```
x="hello learning Python is very easy!!!"
print('h' in x)             # True
print('d' in x)              # False
print('d' not in x)  # True
print('Python' in x)             # True
```

Operator Precedence:

If multiple operators present then which operator will be evaluated first is decided by operator precedence.

Example:

```
print(3+10*2)  # 23
print((3+10)*2)  # 26
```

The following list describes operator precedence in Python .

(1) () → Parenthesis

(2) ** → Exponential Operator

(3) ~, - → Bitwise Complement Operator, Unary Minus Operator

(4) *, /, %, // →Multiplication, Division, Modulo, Floor Division

(5) +, - → Addition, Subtraction

(6) <<, >> → Left and Right Shift

(7) & → Bitwise And

(8) ^ →Bitwise X-OR

(9) | → Bitwise OR

(10) >, >=, <, <=, ==, != → Relational OR Comparison Operators

(11) =, +=, -=, *=... → Assignment Operators

(12) is , is not → Identity Operators

(13) in , not in → Membership operators

(14) not → Logical not

(15) and → Logical and

(16) or → Logical or

Example:

```
a=30
b=20
c=10
d=5
print((a+b)*c/d)          # 100.0
print((a+b)*(c/d))        # 100.0
print(a+(b*c)/d)          # 70.0
```

Python ord() Function:

Unicode values:

A to Z-65 to 90

a to z-97 to 122

The ord() function returns the number representing the unicode code of a specified character.

```
x=ord("h")
print(x)
# output:104
x=ord("H")
print(x)
# output:72
```

Python chr() Function:

The chr() function returns the character that represents the specified unicode.

```
x=chr(97)
print(x)
# output: a
```

Python Math Module

The **math module** in Python is a built-in library that contains a collection of mathematical functions and constants. It is commonly used to perform standard math operations such as rounding, trigonometry, logarithms and more, all with precise and reliable results.

Why do we need Math module ?

1. Provides built-in functions for complex math operations like square root, power and trigonometry.
2. Offers constants like pi and e, useful in scientific and engineering calculations.
3. Improves accuracy and performance over manual calculations or custom functions.
4. Helps in performing logarithmic and exponential operations easily.
5. Supports real-world applications like physics, statistics, geometry and finance.

Importing math module

To work with mathematical constants and functions in Python, you need to import the math module:

import math

Once imported, you can access a variety of constants. Let's explore some of them:

Constants in Math Module

The Python math module provides various values of various constants like pi and tau. We can easily write their values with these constants. The constants provided by the math module are:

- Euler's Number
- Pi
- Tau
- Infinity
- Not a Number (NaN)

Let's see constants in math module with examples:

1. Euler's Number

The **math.e** constant returns the Euler's number: 2.71828182846.

Syntax:

math.e

Example: This prints the value of the mathematical constant **e**.

```
import math
print (math.e)
```

Output

```
2.718281828459045
```

2. Pi

You all must be familiar with pi. The pi is depicted as either 22/7 or 3.14. **math.pi** provides a more precise value for the pi.

Syntax:

math.pi

Example 1: This prints the value of the mathematical constant **pi**.

```
import math
print (math.pi)
```

Output

```
3.141592653589793
```

Example 2: This calculates the area of a circle using **pi**

```
import math
r = 4
pie = math.pi
print(pie * r * r)
```

Output

```
50.26548245743669
```

3. tau

Tau is defined as the ratio of the circumference to the radius of a circle. The **math.tau** constant returns the value tau: 6.283185307179586.

Syntax:

math.tau

Example: This prints the value of the mathematical constant **tau.**

```
import math
print (math.tau)
```

Output

```
6.283185307179586
```

4. Infinity

Infinity basically means something which is never-ending or boundless from both directions i.e. negative and positive. It cannot be depicted by a number. The Python **math.inf** constant returns of positive infinity. For negative infinity use **-math.inf**.

Syntax:

math.inf

Example 1: This prints positive and negative infinity.

```
import math
print (math.inf)
print (-math.inf)
```

Output

```
inf
-inf
```

Example 2: This compares infinity with a large number.

```
import math
print (math.inf > 10e108)
print (-math.inf < -10e108)
```

Output

```
True
True
```

5. NaN Values

The Python **math.nan** constant returns a floating-point nan (Not a Number) value. This value is not a legal number. The nan constant is equivalent to float("nan").

Example: This prints the value of math.nan.

```
import math
print (math.nan)
```

Output

```
Nan
```

Numeric Functions in Math Module

In this section, we will deal with the functions that are used with number theory as well as representation theory such as finding the factorial of a number. We will discuss these numerical functions along with examples and use-cases.

1. Finding the ceiling and the floor value

Ceil value means the smallest integral value greater than the number and the floor value means the greatest integral value smaller than the number. This can be easily calculated using the ceil() and floor() method respectively.

Example: This prints the ceiling and floor value of 2.3.

```
import math
a = 2.3
print ("The ceil of 2.3 is : ", end="")
print (math.ceil(a))
print ("The floor of 2.3 is : ", end="")
print (math.floor(a))
```

Output

```
The ceil of 2.3 is : 3
The floor of 2.3 is : 2
```

2. Finding the factorial of the number

Using the factorial() function we can find the factorial of a number in a single line of the code. An error message is displayed if number is not integral.

Example: This prints the factorial of 5.

```
import math
a = 5
print("The factorial of 5 is : ", end="")
print(math.factorial(a))
```

Output

```
The factorial of 5 is : 120
```

3. Finding the GCD

gcd() function is used to find the greatest common divisor of two numbers passed as the arguments.

Example: This prints the GCD of 15 and 5.

```
import math
a = 15
b = 5
print ("The gcd of 5 and 15 is : ", end="")
print (math.gcd(b, a))
```

Output

```
The gcd of 5 and 15 is : 5
```

4. Finding the absolute value

fabs() function returns the absolute value of the number.

Example: This prints the absolute value of -10

```
import math
a = -10
print ("The absolute value of -10 is : ", end="")
print (math.fabs(a))
```

Output

```
The absolute value of -10 is : 10.0
```

Logarithmic and Power Functions

Power functions can be expressed as x^n where n is the power of x whereas logarithmic functions are considered as the inverse of exponential functions.

1. Finding the power of exp

exp() method is used to calculate the power of e

Example: This prints exponential values of an integer, negative integer and a float.

```
import math
test_int = 4
test_neg_int = -3
test_float = 0.00
print (math.exp(test_int))
print (math.exp(test_neg_int))
print (math.exp(test_float))
```

Output

```
54.598150033144236

0.049787068367863944

1.0
```

2. Finding the power of a number

pow() function computes x**y. This function first converts its arguments into float and then computes the power.

Example: This prints 3 raised to the power of 4.

```
print ("The value of 3**4 is : ",end="")
print (pow(3,4))
```

Output

```
The value of 3**4 is : 81
```

3. Finding the Logarithm

- **log()** function returns the logarithmic value of a with base b. If the base is not mentioned, the computed value is of the natural log.
- **log2(a)** function computes value of log a with base 2. This value is more accurate than the value of the function discussed above.
- **log10(a)** function computes value of log a with base 10. This value is more accurate than the value of the function discussed above.

Example: This prints different logarithmic values.

```
import math
print ("The value of log 2 with base 3 is : ", end="")
print (math.log(2,3))
print ("The value of log2 of 16 is : ", end="")
print (math.log2(16))
print ("The value of log10 of 10000 is : ", end="")
print (math.log10(10000))
```

Output

```
The value of log 2 with base 3 is : 0.6309297535714574

The value of log2 of 16 is : 4.0

The value of log10 of 10000 is : 4.0
```

4. Finding the Square root

sqrt() function returns the square root of the number.

Example: This prints square roots of 0, 4 and 3.5.

```
import math
print(math.sqrt(0))
print(math.sqrt(4))
print(math.sqrt(3.5))
```

Output

```
0.0

2.0

1.8708286933869707
```

Trigonometric and Angular Functions

You all must know about Trigonometric and how it may become difficult to find the values of sine and cosine values of any angle. Math module provides built-in functions to find such values and even to change the values between degrees and radians.

1. Finding sine, cosine and tangent

sin(), **cos()** and **tan()** functions returns the sine, cosine and tangent of value passed as the argument. The value passed in this function should be in radians.

Example: This prints the trigonometric values of π/6.

```
import math
a = math.pi/6
print ("The value of sine of pi/6 is : ", end="")
print (math.sin(a))
print ("The value of cosine of pi/6 is : ", end="")
print (math.cos(a))
print ("The value of tangent of pi/6 is : ", end="")
print (math.tan(a))
```

Output

```
The value of sine of pi/6 is : 0.49999999999999994

The value of cosine of pi/6 is : 0.8660254037844387

The value of tangent of pi/6 is : 0.5773502691896257
```

2. Converting values from degrees to radians and vice versa

- **degrees()** function is used to convert argument value from radians to degrees.
- **radians()** function is used to convert argument value from degrees to radians.

Example: This converts π/6 radians to degrees and 30° to radians.

```
import math
a = math.pi/6
b = 30
print ("The converted value from radians to degrees is : ", end="")
print (math.degrees(a))
print ("The converted value from degrees to radians is : ", end="")
print (math.radians(b))
```

Output

```
The converted value from radians to degrees is : 29.999999999999996

The converted value from degrees to radians is : 0.5235987755982988
```